Survey: Evaluating Prompted, Fine-Tuned, and Reward-Based LLM Judges with JudgeBench

Cameron Goz
University of South Florida
CIS 4930: Intro to LLMs
cgoz@usf.edu

Abstract

Large language models (LLMs) are increasingly used as automatic judges for evaluating model outputs, but their reliability remains uncertain. This survey studies LLM performance as a judge using JudgeBench, a benchmark that evaluates whether a judge can correctly choose the better response from pairwise comparisons with known correct answers. I compare three judge paradigms: prompted judges, fine-tuned judges, and reward-based judges. The evaluated systems include API-based prompted models, a locally run prompted Llama model, a fine-tuned Skywork Critic model, and a Skywork Reward model. Across 350 JudgeBench response pairs, the strongest overall results came from prompted API models, with GPT-4.1-mini and Gemini 2.5 Flash Lite outperforming the local and specialized judge models. Category-level results showed that math was generally the easiest domain, while coding and reasoning exposed larger weaknesses for several judges. Failure-mode analysis revealed that order-swap inconsistency is a major reliability issue for chat-style judges, while the reward model avoided inconsistency but was not the most accurate overall. These results suggest that judge-specific training or reward modeling does not automatically produce stronger evaluation behavior; base model capability, task domain, and consistency under response ordering all matter when using LLMs as judges.

1 Introduction

Large language models (LLMs) are increasingly used not only to generate text, code, and explanations, but also to evaluate the outputs of other models. This "LLM-as-a-judge" approach is attractive because human evaluation is slow, expensive, and difficult to scale. A judge model can quickly compare candidate responses, assign preferences, or score outputs across many examples. As a result, LLM judges are commonly used in model development, benchmark evaluation, and preference-based training pipelines.

However, using an LLM as a judge introduces its own reliability problem. A model that sounds confident is not necessarily correct, and a model that performs well as a chatbot may not always make accurate evaluation decisions. Judge models may prefer longer or more polished answers, miss subtle factual or reasoning errors, or bias their decision based on whichever option was given first. These issues make it important to evaluate judges themselves, rather than assuming that stronger language models automatically provide trustworthy judgments.

This report uses JudgeBench (Tan et al., 2025) as the basis for studying this problem. JudgeBench is a benchmark designed to evaluate LLM-based judges through pairwise comparisons where the correct choice is known. In each trial, a judge is given a question and two candidate responses, and performance is measured by whether the judge selects the objectively correct one. This setup makes JudgeBench useful for measuring judge accuracy in a more controlled way than open-ended preference evaluation.

The survey focuses on three common judge paradigms. The first is prompted judging, where a general-purpose LLM is given a judging prompt and asked to choose the better response. The second is fine-tuned judging, where a model has been trained specifically for evaluation or critique tasks. The third is reward-based judging, where a reward model assigns scores to candidate responses and the higher-scoring one is selected. Comparing these paradigms helps show whether specialized judging methods outperform strong general models.

The experiments evaluate several prompted, fine-tuned, and reward-based judges on a 350-pair JudgeBench evaluation set across knowledge, reasoning, math, and coding tasks. Beyond accuracy, the report examines failure modes such as incorrect

judgments and order-swap inconsistency. One limitation worth noting is that the response pairs come from a GPT-4o-based dataset, which may affect interpretation when testing related OpenAI models. While this does not make the results invalid, comparisons involving those models should be read with potential model-family overlap in mind.

2 Background

2.1 Prompted Judges

A prompted judge uses an existing general-purpose language model with a structured evaluation prompt, requiring no additional training or fine-tuning. In the Arena-Hard format (Li et al., 2024) used in this study, the model receives a question and two candidate responses and is instructed to output a direct preference: A is better, B is better, or they are tied. The judgment relies entirely on the model's pretrained reasoning and domain knowledge. Because the approach is prompt-only, judge quality scales directly with the capability of the underlying model, and any model accessible via API or local inference can serve as a judge with no modification.

The primary known failure mode of prompted judges is position bias: a tendency to prefer whichever response appears first in the context, independent of actual quality. JudgeBench addresses this by evaluating each pair in two orders. A judge that reverses its verdict when only the presentation order changes is flagged as order-swap inconsistent, a reliability failure distinct from simply selecting the wrong answer.

2.2 Fine-Tuned Judges

A fine-tuned judge starts from a pretrained model and is further trained on labeled preference or evaluation data, with the goal of teaching the model evaluation-specific behavior beyond what general pretraining provides. Skywork-Critic-Llama-3.1-8B (Tu et al., 2024), the fine-tuned judge evaluated here, was trained specifically for critique and pairwise comparison tasks. It uses a dedicated evaluation prompt format it was trained to produce, outputting a structured verdict of `[[A]]` or `[[B]]` rather than an open-ended response.

Fine-tuned judges occupy a middle ground between prompted judges and reward models: they generate natural language verdicts like prompted judges, but their evaluation behavior is shaped by task-specific supervision. Whether that training advantage outweighs the benefit of using a larger general-purpose model is one of the central empirical questions this study addresses.

2.3 Reward Models

Reward models were originally developed as a component of reinforcement learning from human feedback (RLHF). A reward model is trained on human-labeled preference pairs to assign a scalar score to any given response, where higher scores indicate outputs more aligned with human preferences. This signal is then used to guide policy optimization of the target language model during RLHF training.

In a pairwise evaluation context, the reward model is repurposed as a judge: both candidate responses are scored independently, and the higher-scoring response is selected. Because scoring is response-level rather than comparison-level, the order in which responses appear is irrelevant, eliminating position bias and making order-swap inconsistency structurally impossible. This is the key mechanical difference from chat-style judges. The tradeoff is transparency: unlike prompted and fine-tuned judges, the reward model produces no explanatory text, making individual judgment decisions opaque.

3 Experimental Setup

3.1 Benchmark and Task

The experiments use the 350-pair GPT-4o JudgeBench evaluation set spanning four categories: knowledge (`mmlu-pro`), reasoning (`livebench-reasoning`), mathematics (`livebench-math`), and coding (`livecodebench`), with 154, 98, 56, and 42 pairs respectively. Each example contains a source question, two candidate responses generated by GPT-4o, and a ground-truth label identifying the objectively correct response. The judge receives the question and both responses and must output which it considers better.

Each pair is presented in two orderings: original and order-swapped. All chat-style judges are evaluated in both passes; a verdict reversal between them is counted as an order-swap inconsistency. The reward model was run in single-pass mode, as its response-level scoring is inherently order-independent.

3.2 Models Evaluated

The evaluated judges were selected to represent three approaches to LLM-based evaluation:

prompted judges, a fine-tuned judge, and a reward model. Table 1 lists all models.

The prompted group includes GPT-4.1-mini (OpenAI, 2025), GPT-4o-mini (OpenAI, 2024), Gemini 2.5 Flash Lite (Gemini Team, 2025), and Llama-3.1-8B-Instruct (Grattafiori et al., 2024). All four received the same Arena-Hard judging prompt (Li et al., 2024), so differences in performance reflect the underlying model rather than prompting strategy. GPT-4o-mini and Llama-3.1-8B-Instruct also span the API-vs.-local axis, allowing a practical comparison between the two deployment modes.

The fine-tuned judge is Skywork-Critic-Llama-3.1-8B (Tu et al., 2024), a model trained specifically for critique and evaluation tasks, run locally through a vLLM-served endpoint. Including it tests whether a model tuned for evaluation outperforms general-purpose prompted judges.

The reward-based judge is Skywork-Reward-Llama-3.1-8B (Liu et al., 2024), also run locally. Unlike chat-style judges, it scores each candidate response individually; the higher-scoring response is taken as the selected answer. This structural difference makes it useful for examining whether scoring-based selection behaves differently from verbal pairwise judgment.

Model	Judge Type	Access
GPT-4.1-mini	Prompted	API
GPT-4o-mini	Prompted	API
Gemini 2.5 Flash Lite	Prompted	API
Llama-3.1-8B-Instruct	Prompted	Local
Skywork-Critic-Llama-3.1-8B	Fine-tuned	Local
Skywork-Reward-Llama-3.1-8B	Reward model	Local

Table 1: Models evaluated in this study, organized by judge type and access method.

3.3 Judge Configuration

All prompted judges used the Arena-Hard prompt format. The model receives the original question and two candidate responses and is instructed to choose which response is better. The prompt template was identical across all prompted models. Chat-style judge outputs were parsed into a pairwise verdict of A>B, B>A, or A=B. Ties were counted as incorrect since each JudgeBench pair has a definite ground-truth label.

The fine-tuned critic (Skywork-Critic) was evaluated using its own prompt format rather than Arena-Hard. The reward model (Skywork-Reward) received each response individually, and the higher-scored response was taken as the selected answer.

API-based models were called through their respective provider APIs with default parameters. Local models were served via vLLM on my local machine equipped with an RTX 3090 Ti GPU. All judges were evaluated on the same 350 response pairs, and outputs were saved as JSONL files for post-processing.

3.4 Evaluation Metrics

Three metrics are used throughout the evaluation.

Pairwise accuracy is the primary metric: the percentage of response pairs where the judge selects the correct response. For chat-style judges evaluated in two-pass mode, a pair is counted as correct only if the judge selects the correct underlying response in both orderings.

Inconsistency rate is the percentage of pairs where the judge reverses its verdict between the two orderings. A reliable judge should select the same underlying response regardless of whether it appears as response A or response B.

Failure rate is the combined percentage of incorrect and inconsistent pairs, providing an overall picture of how often a judge fails to produce a stable correct answer.

3.5 Experimental Questions

Three comparisons structure the analysis.

Prompted judge comparison: Given that all prompted judges use the same Arena-Hard prompt, how much does the choice of underlying model affect judging quality? This targets GPT-4.1-mini, GPT-4o-mini, Gemini 2.5 Flash Lite, and Llama-3.1-8B-Instruct.

Cross-paradigm comparison: How do prompted judges, the fine-tuned critic, and the reward model compare in accuracy and failure modes? This captures whether specialized judging methods offer advantages over strong general-purpose prompted models.

Category-level analysis: Do judge rankings and failure patterns change across task categories? This examines whether knowledge, reasoning, math, and coding stress different judge capabilities.

4 Results

4.1 Overall Performance

Table 2 reports overall accuracy, incorrect rate, inconsistency rate, and failure rate for all six judges.

GPT-4.1-mini achieves the highest overall accuracy at 70.6%, followed closely by Gemini 2.5 Flash Lite at 69.1%. Both API-based models are substantially ahead of the remaining four judges. The Skywork Reward model places third at 62.0%, the Skywork Critic fourth at 54.0%, GPT-4o-mini fifth at 48.6%, and Llama-3.1-8B last at 39.4%.

Llama-3.1-8B has the highest overall inconsistency rate (34.9%), meaning more than one in three pairs produced a flipped verdict on order swap. GPT-4o-mini has a high combined failure rate (51.4%), split nearly evenly between incorrect and inconsistent outcomes. The reward model eliminates inconsistency by design, but trades this for the highest incorrect rate among non-Llama models (38.0%). Figure 1 shows this outcome breakdown across all six judges.

Model	Acc	Incorr	Incons	Fail
GPT-4.1-mini	70.6%	11.7%	17.7%	29.4%
Gemini 2.5 FL	69.1%	9.7%	21.1%	30.9%
Skywork-Reward	62.0%	38.0%	0.0%	38.0%
Skywork-Critic	54.0%	28.0%	18.0%	46.0%
GPT-4o-mini	48.6%	26.3%	25.1%	51.4%
Llama-3.1-8B	39.4%	25.7%	34.9%	60.6%

Table 2: Overall results for all six judges. Acc = pairwise accuracy; Incorr = incorrect rate; Incons = inconsistency rate; Fail = combined failure rate. All values are percentages of the 350 evaluated pairs.

4.2 Category-Level Performance

Table 3 shows accuracy broken down by category for all six judges. Mathematics is the easiest category for five of the six judges: GPT-4.1-mini and Gemini 2.5 Flash Lite reach 80.4% and 82.1% respectively, while even Llama-3.1-8B rises to 44.6% on math versus 33.3% on coding. Coding (`livecodebench`) is particularly difficult for the Skywork Critic (31.0%) and Llama-3.1-8B (33.3%), and is the weakest category for the reward model (50.0%).

The reward model is notably uneven: it performs well on math (76.8%) and reasoning (63.3%) but drops sharply on coding. The Skywork Critic similarly collapses on coding relative to its performance elsewhere, suggesting that its fine-tuning did not generalize well to code evaluation. The top two prompted models maintain more stable performance across categories. Figure 2 illustrates these differences across all four domains.

Model	Know	Reas	Math	Code
GPT-4.1-mini	65.6%	72.4%	80.4%	71.4%
Gemini 2.5 FL	65.6%	67.3%	82.1%	69.0%
Skywork-Reward	59.1%	63.3%	76.8%	50.0%
Skywork-Critic	52.6%	54.1%	75.0%	31.0%
GPT-4o-mini	47.4%	43.9%	66.1%	40.5%
Llama-3.1-8B	38.3%	40.8%	44.6%	33.3%

Table 3: Category-level pairwise accuracy. Know = `mmlu-pro`; Reas = `livebench-reasoning`; Math = `livebench-math`; Code = `livecodebench`.

4.3 Cross-Paradigm Comparison

Table 4 compares the three judge paradigms by averaging accuracy across their constituent models. The prompted paradigm average (56.9%) is pulled down by GPT-4o-mini and Llama-3.1-8B; the two strongest prompted models individually exceed the reward model. Across paradigms, the reward model (62.0%) outperforms the fine-tuned critic (54.0%) on every category, with the largest gap on coding (50.0% vs. 31.0%).

Inconsistency rates differ structurally. The reward model has 0% inconsistency by design. The fine-tuned critic (18.0%) is lower than the prompted group average (24.7%), which may reflect its exposure to structured evaluation during training. Within the prompted group, inconsistency ranges from 17.7% (GPT-4.1-mini) to 34.9% (Llama-3.1-8B).

The reward model's scalar score margin (the absolute difference in scores assigned to the two responses) provides further insight into paradigm-level behavior. Correct judgments had a mean margin of 10.00 versus 6.13 for incorrect judgments, indicating the model is generally more decisive when right than when wrong. The gap is largest on math pairs (12.35 correct vs. 4.82 incorrect), consistent with math being the easiest category overall. On coding, the margins converge almost entirely (7.59 correct vs. 7.38 incorrect), meaning the reward model is no more confident on its correct coding judgments than its incorrect ones, a quantitative explanation for its near-random coding accuracy. Figure 7 in the appendix shows these margins broken down by category.

Within the prompted paradigm, inter-model agreement reveals additional structure. Only 19.7% of pairs were answered correctly by all four prompted models unanimously, and 10.3% were answered incorrectly by all four, meaning these pairs defeat even the strongest judges regardless

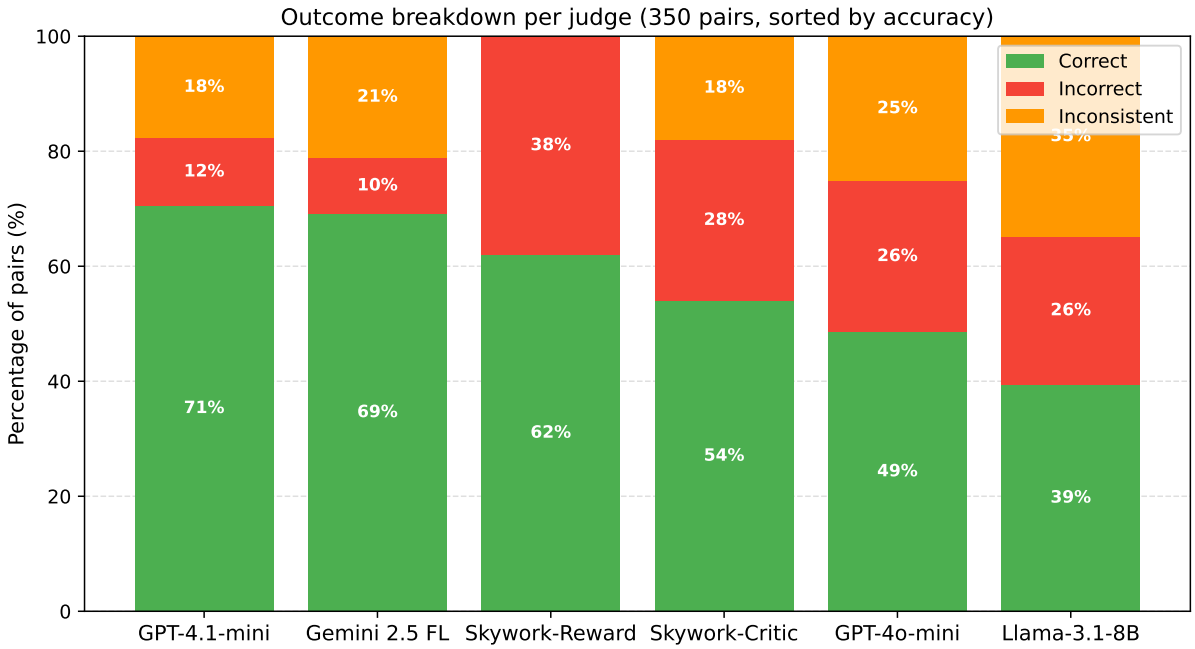

Figure 1: Outcome breakdown per judge across 350 evaluated pairs, sorted by overall accuracy. Each bar shows the proportion of pairs classified as correct (green), incorrect (red), and inconsistent (orange). The reward model has no inconsistency by design.

of model choice. The pairwise agreement between GPT-4.1-mini and Gemini 2.5 Flash Lite was 58.0%, reflecting their similar capability tier. Llama-3.1-8B agreed with each of the other three models at only 37–46%, confirming that its failures are not simply correlated with those of stronger models. Figure 6 in the appendix shows the full pairwise agreement matrix, and Figure 3 breaks down how often each consensus tier occurs across the 350 pairs.

Paradigm	Know	Reas	Math	Code	Overall
Prompted	54.2%	56.1%	68.3%	53.6%	56.9%
Fine-tuned	52.6%	54.1%	75.0%	31.0%	54.0%
Reward	59.1%	63.3%	76.8%	50.0%	62.0%

Table 4: Average category-level accuracy by judge paradigm. The prompted average includes all four prompted models.

4.4 Failure Mode Analysis

Order-swap inconsistency is the dominant failure mode for several chat-style judges. For Llama-3.1-8B, inconsistency alone accounts for 34.9% of all pairs, exceeding its incorrect rate of 25.7%. For Gemini 2.5 Flash Lite and GPT-4.1-mini, incorrect and inconsistent outcomes are more balanced at roughly 10–22% each. GPT-4o-mini shows high rates of both: 26.3% incorrect and 25.1% inconsistent, for a combined failure rate of 51.4%.

Inconsistency is also category-dependent. Coding has the highest inconsistency rate for both Gemini (26.2%) and GPT-4.1-mini (26.2%), despite both models achieving relatively high accuracy in that category. This suggests that coding pairs are genuinely difficult to judge consistently, even for capable models.

The reward model trades inconsistency for a uniformly higher incorrect rate. On coding it reaches exactly 50% incorrect (essentially guessing) and on knowledge its incorrect rate (40.9%) is the highest of any model on that category. This indicates that scalar-scoring avoids ordering bias but does not produce more accurate pairwise preferences on hard objective tasks. Figure 4 shows the full inconsistency breakdown by judge and category.

4.5 Qualitative Error Analysis

Quantitative failure rates describe how often judges fail but not how or why. Examining specific pairs reveals patterns that aggregate numbers do not capture.

Universally hard pairs: A subset of pairs defeat all six judges. One representative reasoning pair asks the judge to evaluate two responses to a multi-attribute constraint satisfaction puzzle, where one response derives the correct answer and the other

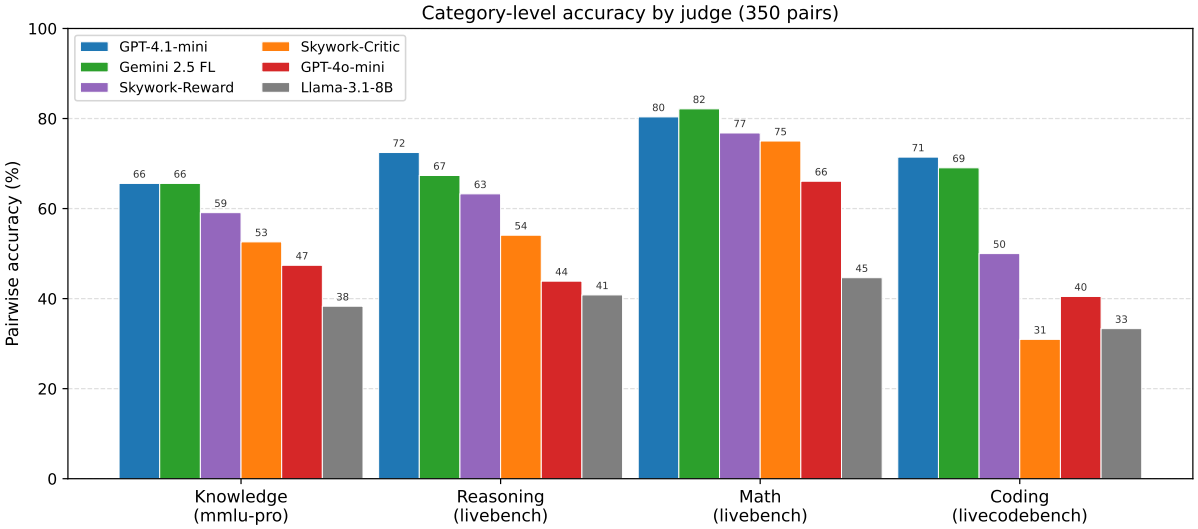

Figure 2: Category-level pairwise accuracy for all six judges across the four JudgeBench task domains. Models are ordered left to right by overall accuracy within each group.

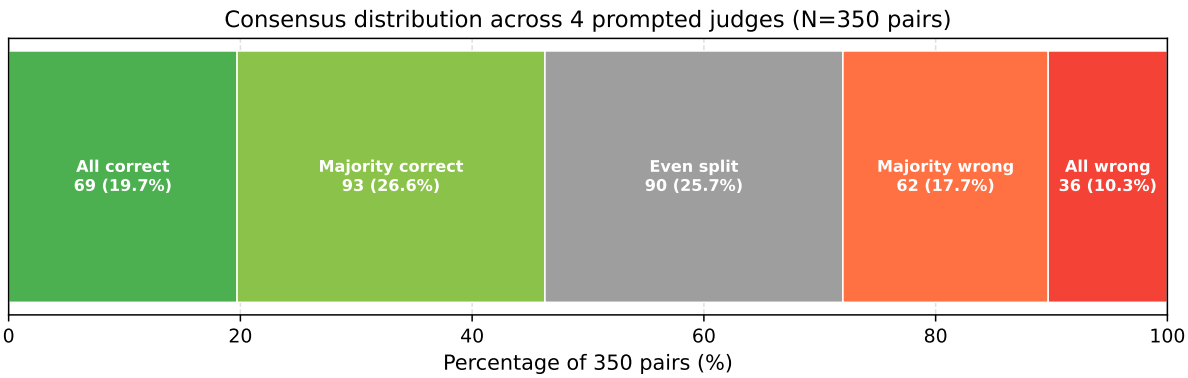

Figure 3: Consensus distribution across the four prompted judges. Each segment represents one difficulty tier: pairs all judges answered correctly, pairs where a majority was correct, even splits, pairs where only one judge was correct, and pairs no judge answered correctly.

does not. Every model in the study either selected the incorrect response or produced an inconsistent verdict. A knowledge pair involving the legal voluntariness of a confession showed the same pattern: all six judges produced incorrect or inconsistent judgments regardless of paradigm or model size. These pairs likely require domain depth or multi-step verification that exceeds what any judge here reliably performs. They account for the 10.3% of pairs that all four prompted models failed unanimously, suggesting a hard floor on judge accuracy that is not specific to any one paradigm.

Inconsistency as a distinct failure mode: A counterintuitive pattern emerges across several coding pairs: a judge can be consistently wrong without being inconsistent, and appear stable while actually being order-sensitive. On one coding pair, Gemini 2.5 Flash Lite returned a tie in both passes, a stable but incorrect result, since the ground-truth label identifies a clear winner. GPT-4.1-mini returned the same raw verdict in both passes yet was still flagged as inconsistent, because in the swapped-order pass that same verdict corresponds to selecting the opposite underlying response. These cases illustrate that consistency and accuracy are independent properties: a judge can fail reliably, or appear stable while remaining sensitive to presentation order in a way that only the two-pass structure reveals.

Reward model opacity: The reward model's failures differ structurally from those of chat-style judges. It produces no explanatory text, so its errors are opaque by design. On coding pairs it scores correctly just 50% of the time, indistinguishable

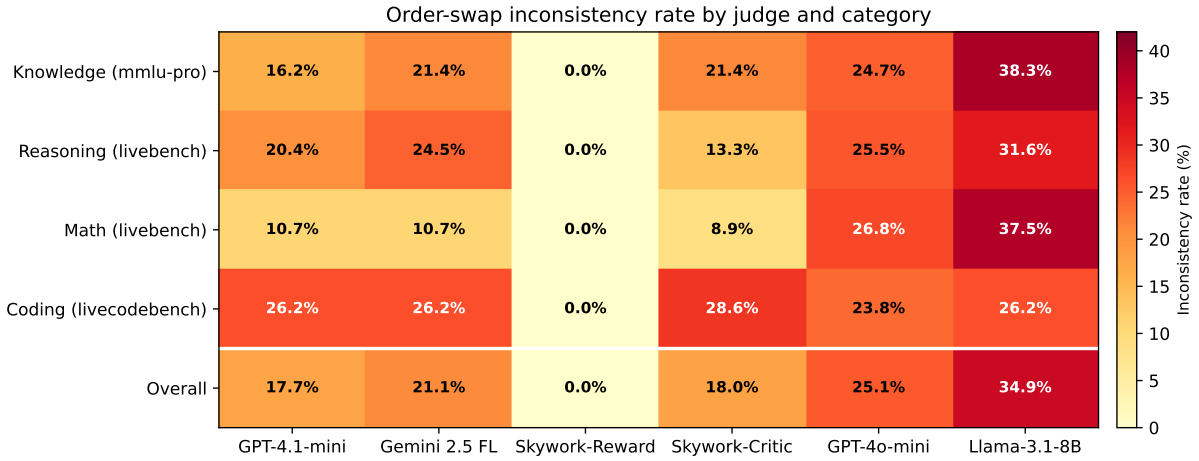

Figure 4: Order-swap inconsistency rate (%) by judge and task category. The reward model has 0% inconsistency by design. Deeper shading indicates higher inconsistency; the horizontal line separates per-category rows from the Overall summary row.

from random selection. Whether this reflects miscalibrated scoring on code-style responses, a distributional mismatch with GPT-4o-generated text, or an inherent limitation of scalar scoring on objective tasks cannot be determined from the output alone. This opacity is a practical constraint when using reward models as standalone evaluators rather than as training signals.

5 Discussion

The most striking finding is the gap between the two top API-based prompted judges and all locally run models. GPT-4.1-mini (70.6%) and Gemini 2.5 Flash Lite (69.1%) outperform the remaining four models by large margins. When the judging prompt is fixed, the primary driver of judge quality appears to be the underlying model's capability. A simple Arena-Hard prompt applied to a strong model is sufficient to produce competitive judging behavior.

GPT-4o-mini underperforms relative to its API status, landing at 48.6%, below even the reward model. Although it is a capable general-purpose model, this result suggests that API access alone does not guarantee good judging. Model generation and calibration matter: GPT-4o-mini's high inconsistency (25.1%) combined with its high incorrect rate (26.3%) points to a model that is both uncertain and unstable as a judge.

The fine-tuned critic does not outperform the reward model despite being explicitly trained for evaluation tasks. Its 54.0% overall accuracy falls short of the reward model's 62.0%, and its coding accuracy (31.0%) is the lowest of any non-Llama model. Fine-tuning for critique appears not to transfer well to code evaluation, and the reward model, which was never trained for pairwise judgment, nonetheless achieves a better overall profile.

Llama-3.1-8B-Instruct demonstrates the practical limitations of small local models as judges. At 39.4% overall accuracy and a 34.9% inconsistency rate, it is only marginally above random for a binary choice task and is frequently reversed by order swaps. Deploying a small local model as a judge with a structured prompt may yield unreliable evaluations regardless of prompting strategy.

Mathematics is consistently the easiest category, which reflects the structure of JudgeBench math pairs (sourced from LiveBench): one response typically contains a correct derivation or boxed answer while the other does not, making the better response structurally identifiable even for weaker models. Coding is the most problematic category overall, with the highest inconsistency rates and the lowest accuracy for several judges. Evaluating code correctness requires reasoning about program behavior, which is harder to assess from verbal pairwise judgment than mathematical or factual correctness.

A key limitation is that all response pairs were generated by GPT-4o, while two evaluated judges (GPT-4o-mini, GPT-4.1-mini) are in the same model family. These models may exhibit implicit preferences for or familiarity with GPT-4o-style responses, which could inflate their accuracy on this benchmark. Separating this effect would require evaluating on Claude- or Llama-generated response sets. Note that JudgeBench provides a 250

response pair dataset created by Claude 3.5 Sonnet. Additional limitations include the small evaluation set (350 pairs), the use of a single judging prompt across all prompted models, and the absence of repeated trials beyond the two-pass order-swap structure.

6 Resources

Local experiments were run on my local desktop with an NVIDIA RTX 3090 Ti GPU (24 GB VRAM), which supported inference for Llama-3.1-8B-Instruct, Skywork-Critic-Llama-3.1-8B, and Skywork-Reward-Llama-3.1-8B via vLLM. API experiments used the OpenAI API (GPT-4.1-mini, GPT-4o-mini) and the Google Gemini API (Gemini 2.5 Flash Lite). OpenAI API costed $2.20 between both models, and Google Gemini $1.35. The local setup used Python, PyTorch, Hugging Face Transformers, and vLLM for model serving.

7 Code

All code used to run these experiments is available at:

`https://github.com/gozcam/Survey-of-JudgeBench`

The repository includes runner scripts for each model (under `scripts/`), the JudgeBench evaluation framework (under `third_party/judgebench/`), pilot and full-run datasets (under `data/`), saved judge outputs (under `outputs/`), and analysis scripts that produce the summary statistics reported in this paper. A `README.md` at the repository root provides step-by-step instructions for setting up the environment and reproducing each experiment.

8 Conclusion

This survey used JudgeBench to compare six LLM judges spanning three paradigms: prompted, fine-tuned, and reward-based. They were tested across 350 response pairs in four task categories. The strongest judges were two API-based prompted models, GPT-4.1-mini and Gemini 2.5 Flash Lite, each achieving approximately 70% accuracy. Locally run models, including a fine-tuned critic and a reward model, trailed behind; the reward model nonetheless outperformed the fine-tuned critic despite having no explicit training for pairwise evaluation. Order-swap inconsistency was the dominant failure mode for chat-style judges, particularly for smaller models. Taken together, these results suggest that judge quality is driven primarily by base model capability, and that specialized training or alternative scoring paradigms do not automatically close that gap. For practitioners building LLM-based evaluation pipelines, this study highlights the importance of testing judge reliability directly rather than assuming that specialization or paradigm choice translates to accuracy.

References

Gemini Team. 2025. Gemini 2.5 flash lite.

Aaron Grattafiori and 1 others. 2024. The llama 3 herd of models. *Preprint*, arXiv:2407.21783.

Tianle Li, Wei-Lin Chiang, Evan Frick, Lisa Dunlap, Tianhao Wu, Banghua Zhu, Joseph E. Gonzalez, and Ion Stoica. 2024. From crowdsourced data to high-quality benchmarks: Arena-hard and benchbuilder pipeline. *Preprint*, arXiv:2406.11939.

Chris Yuhao Liu, Liang Zeng, Jiacai Liu, Rui Yan, Jujie He, Chaojie Wang, Shuicheng Yan, Yang Liu, and Yahui Zhou. 2024. Skywork-reward: Bag of tricks for reward modeling in llms. *Preprint*, arXiv:2410.18451.

OpenAI. 2024. Gpt-4o mini: advancing cost-efficient intelligence.

OpenAI. 2025. Introducing gpt-4.1 in the api. `https://openai.com/index/gpt-4-1/`.

Sijun Tan, Siyuan Zhuang, Kyle Montgomery, William Y. Tang, Alejandro Cuadron, Chenguang Wang, Raluca Ada Popa, and Ion Stoica. 2025. Judgebench: A benchmark for evaluating llm-based judges. *Preprint*, arXiv:2410.12784.

Shiwen Tu, Liang Zhao, Chris Yuhao Liu, Liang Zeng, and Yang Liu. 2024. Skywork critic model series. `https://huggingface.co/Skywork`.

A Supplementary Figures

Model	Know	Reas	Math	Code
GPT-4.1-mini	18.2%	7.1%	8.9%	2.4%
Gemini 2.5 FL	13.0%	8.2%	7.1%	4.8%
Skywork-Reward	40.9%	36.7%	23.2%	50.0%
Skywork-Critic	26.0%	32.7%	16.1%	40.5%
GPT-4o-mini	27.9%	30.6%	7.1%	35.7%
Llama-3.1-8B	23.4%	27.6%	17.9%	40.5%

Table 5: Incorrect rate (%) per judge and category. Know = `mmlu-pro`; Reas = `livebench-reasoning`; Math = `livebench-math`; Code = `livecodebench`. Values are percentages of all pairs in that category.

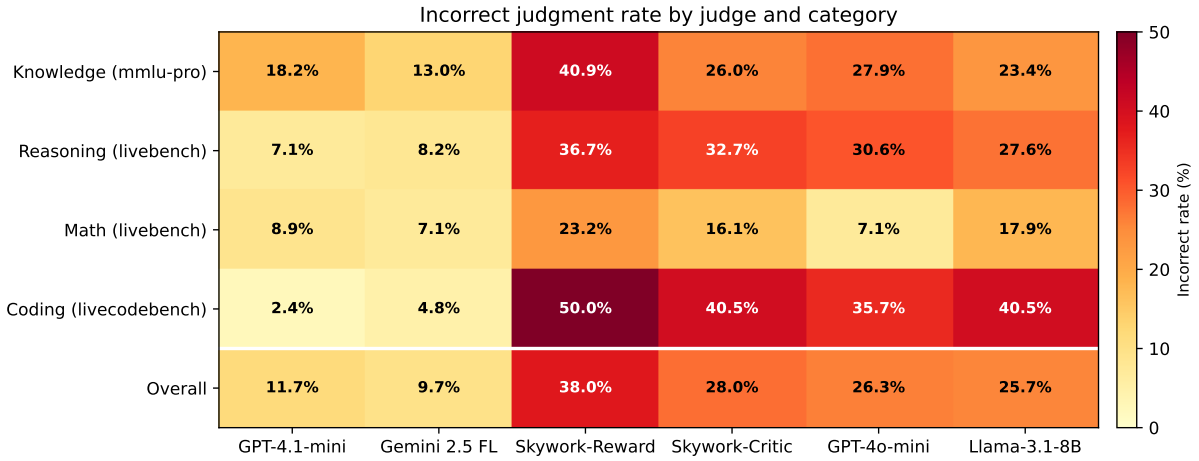

Figure 5: Incorrect judgment rate (%) by judge and task category. Unlike inconsistency, an incorrect outcome means the judge selected the wrong response in both orderings. The reward model's incorrect rates are substantially higher than those of the top prompted models, particularly on coding (50.0%) and knowledge (40.9%). Compare with Figure 4 to see how incorrect and inconsistent failures trade off across models.

Model	Know	Reas	Math	Code
GPT-4.1-mini	16.2%	20.4%	10.7%	26.2%
Gemini 2.5 FL	21.4%	24.5%	10.7%	26.2%
Skywork-Reward	0.0%	0.0%	0.0%	0.0%
Skywork-Critic	21.4%	13.3%	8.9%	28.6%
GPT-4o-mini	24.7%	25.5%	26.8%	23.8%
Llama-3.1-8B	38.3%	31.6%	37.5%	26.2%

Table 6: Inconsistency rate (%) per judge and category. The reward model has 0% across all categories by design. Llama-3.1-8B has the highest inconsistency on knowledge and math; all prompted models show elevated inconsistency on coding ($\geq$20%).

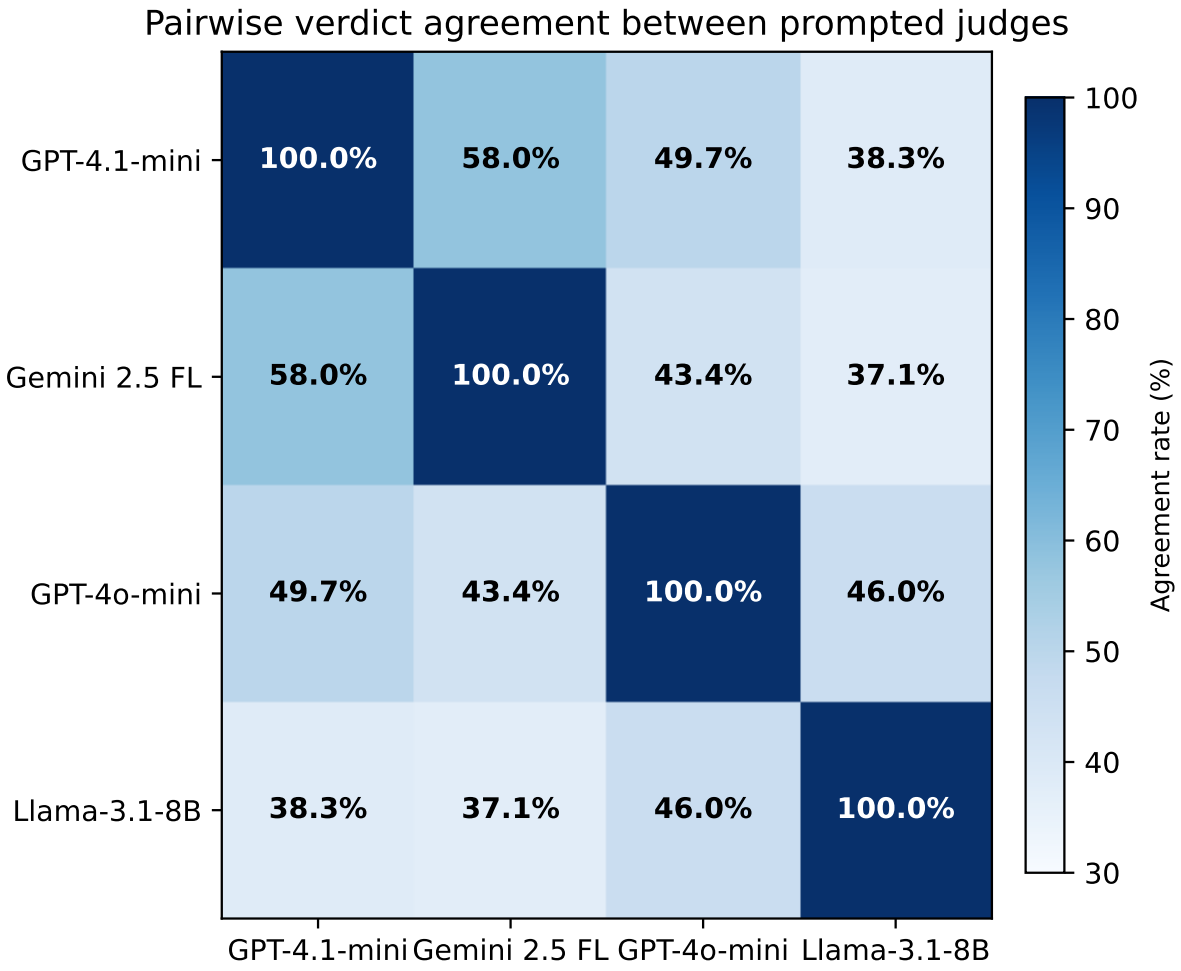

Figure 6: Pairwise verdict agreement between the four prompted judges. Each cell shows the percentage of 350 pairs on which both models gave the same final verdict. The GPT-4.1-mini and Gemini 2.5 Flash Lite cluster (58.0%) reflects their similar capability tier, while Llama-3.1-8B's low agreement with all others (37–46%) confirms that its errors are largely independent of the stronger models.

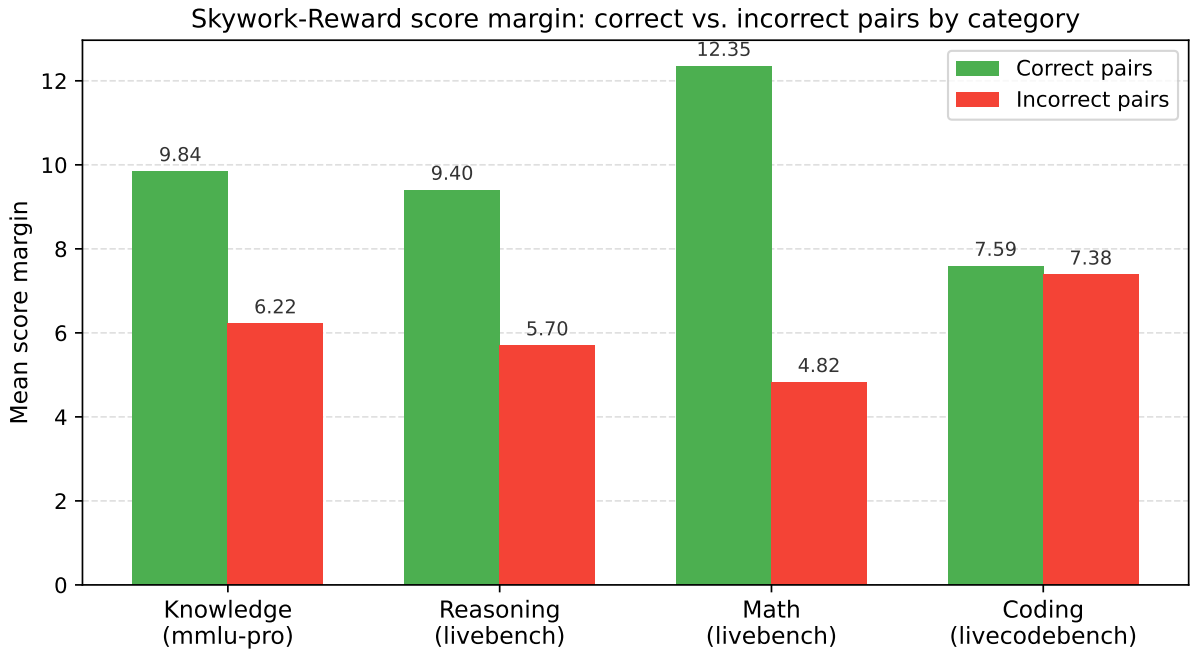

Figure 7: Mean score margin (absolute difference between the two candidate scores) for the Skywork Reward model, split by whether the judgment was correct or incorrect. A larger margin indicates greater model confidence. The near-identical margins on coding pairs (7.59 correct vs. 7.38 incorrect) show the reward model is no more confident when right than when wrong on that category.